

CSC1031

OOP

Misha

Task 10: GameLobby.Java

Define behavior

Implement everything
that will be the same for
all player types

Put here all the
differences and
specifics of each type

interface

Player

Abstract class

AbstractPlayer

Concrete
classes

AIPlaer

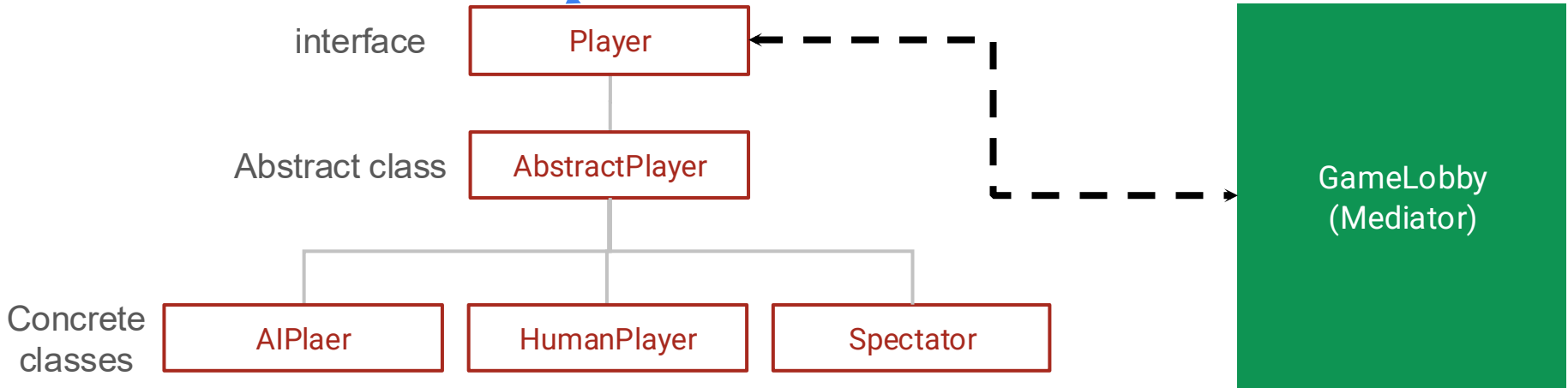
HumanPlayer

Spectator

```
List<Player> players = new ArrayList<>();
```

Place where we declared
(not implemented)
communication methods
with mediator

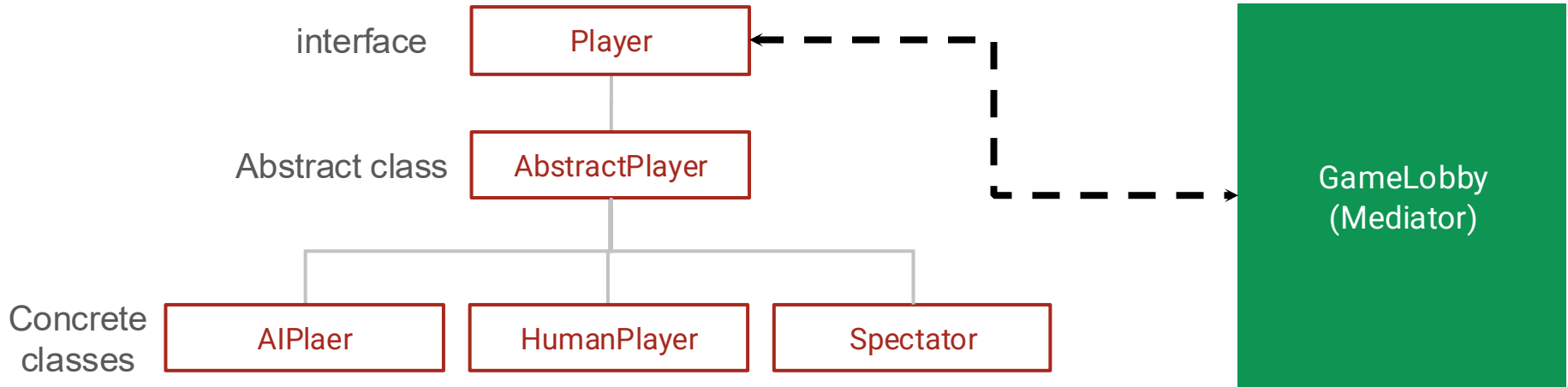
GameLobby can
communicate with
everything that inherits
Player interface



```
List<Player> players = new ArrayList<>();
```

```
List<AbstractPlayer> players = new ArrayList<>();  
List<AIPlayer> players_AI = new ArrayList<>();  
List<HumanPlayer> players_ppl = new ArrayList<>();
```

Also
possible,
but why?



We can store **any type of player**, and if we add a new player type, we **don't need to modify anything**.

```
List<Player> players = new ArrayList<>();
```

What Actually Happens in Memory?

We're saying: "This list will hold objects that follow the **Player contract**."

Each object inside the list is an instance of HumanPlayer, AIPlayer, etc.

Java knows these objects are Players and will only **allow methods defined in Player to be called**.

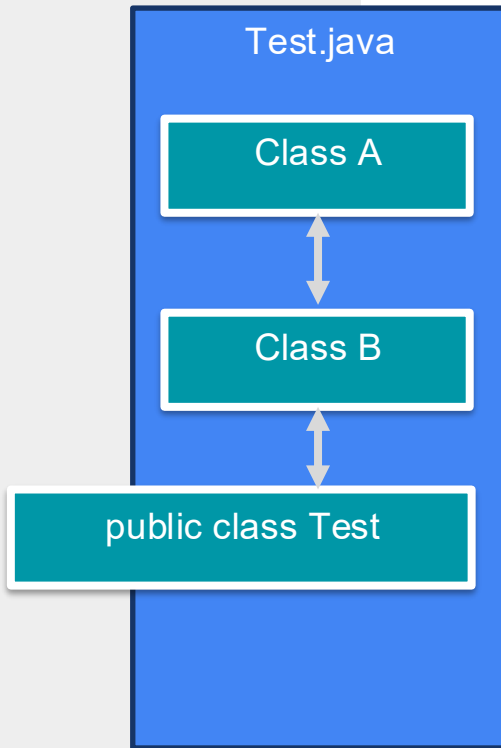
The GameLobby mediator doesn't need to know whether a player is human or AI - it just needs a list of players to manage.

```
List<Player> players = new ArrayList<>();
```

Outer and Inner Classes

An **outer class** is a **regular class** in Java that is not nested inside another class.

```
class A {  
    int x = 10; // Instance variable  
}  
  
class B {  
    int x = 10;  
    A obj_in_B;  
  
    void f(A a){  
        a.x = 12;  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        A obj1 = new A(); // Create an instance of A  
        B obj2 = new B();  
    }  
}
```



Characteristics of an **Outer Class**

Can contain variables and methods.

Can be **instantiated directly**.

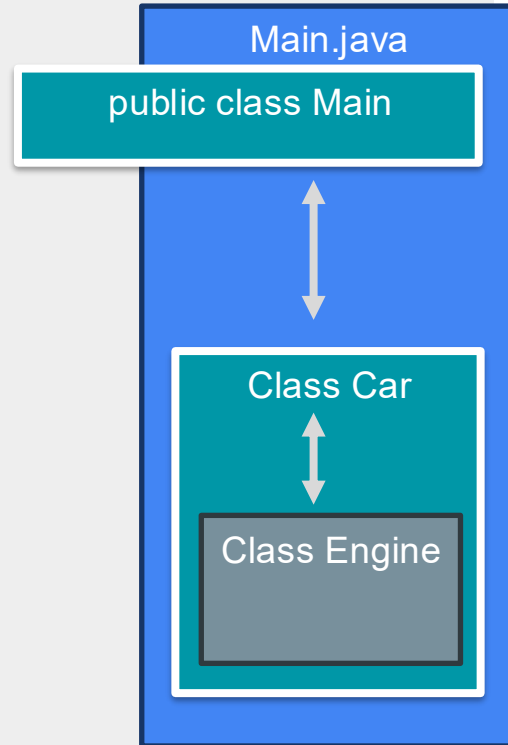
Can be public or package-private (default).

Cannot be static, since it is the top-level class.

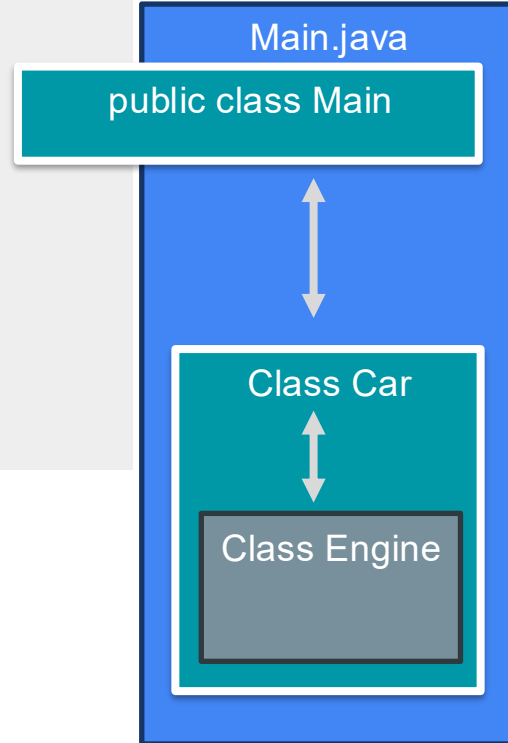
What is an Inner Class?

An inner class is a **class inside another class**.

```
class Car {  
    String carName;  
    String carType;  
  
    // private method  
    private String getCarName() {  
        return this.carName;  
    }  
  
    // inner class  
    class Engine {  
        String engineType;  
  
        void setEngine() {  
            // Accessing the carType property of Car  
            if(Car.this.carType.equals("4WD")){  
                // Invoking method getCarName() of Car  
                if(Car.this.getCarName().equals("Crysler")) {
```




```
public class Main {  
    public static void main(String[] args) {  
  
        // create an object of the outer class Car  
        Car car1 = new Car("Mazda", "8WD");  
  
        // create an object of inner class using the outer class  
        Car.Engine engine = car1.new Engine();  
    }  
}
```



Characteristics of an Inner Class

Nested inside an outer class.

Has access to outer class variables and methods.

Requires an instance of the outer class to be created.

Used for logically grouping related code.

Inner classes are useful when a class is only relevant to another class and should not be used outside.

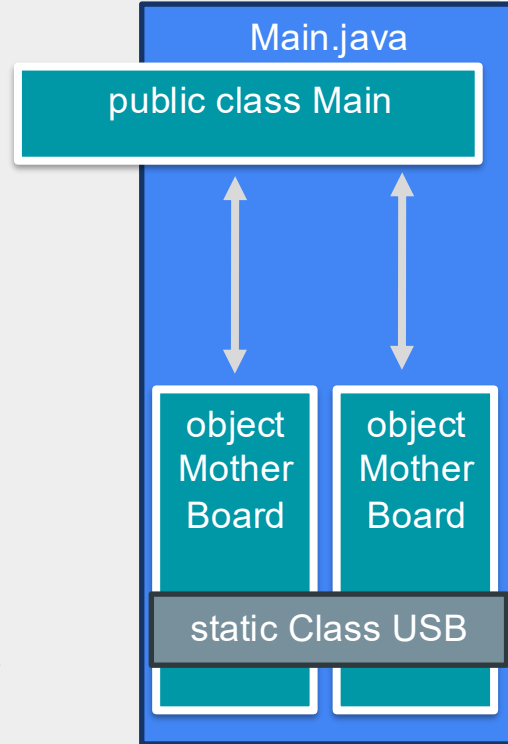
What Happens If the **Inner Class is Static**?

If an inner class is declared as static, it **no longer belongs to an instance** of the outer class. Instead, it behaves like an independent class inside the outer class, meaning:

- It does NOT need an instance of the outer class to be created.
- It CANNOT access non-static members (instance variables/methods) of the outer class.
- It can be instantiated just like a normal class, using `Outer.StaticInner`.

What Happens If the **Inner Class is Static**? (nested class)

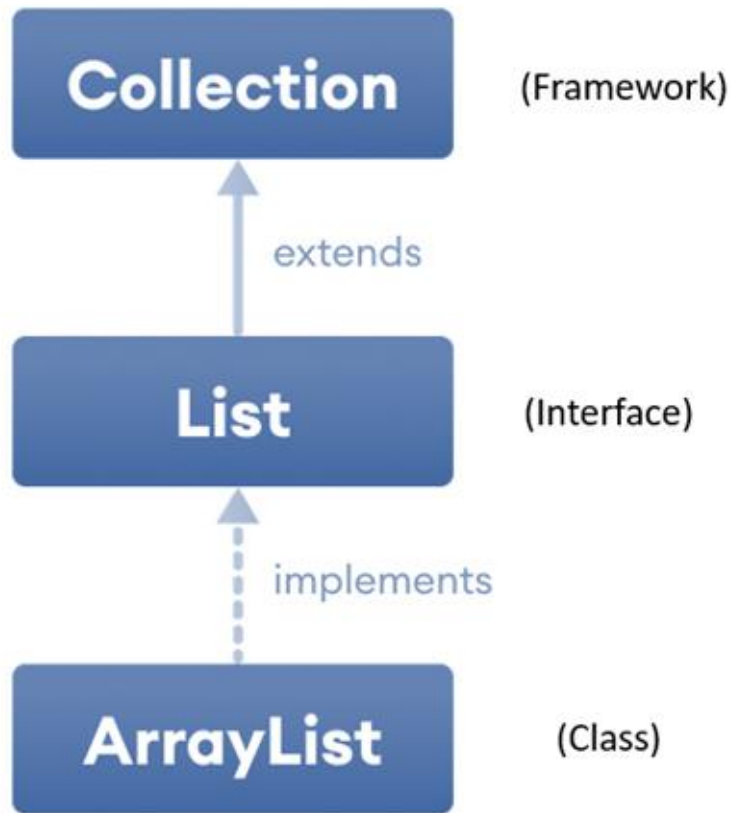
```
class MotherBoard {  
  
    // static nested class  
    static class USB{  
        int usb3 = 1;  
        int getUSB(){  
            return usb3;  
        }  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        MotherBoard.USB usb = new MotherBoard.USB();  
        System.out.println("Total Ports = " + usb.getUSB());  
    }  
}
```



Difference Between List and ArrayList

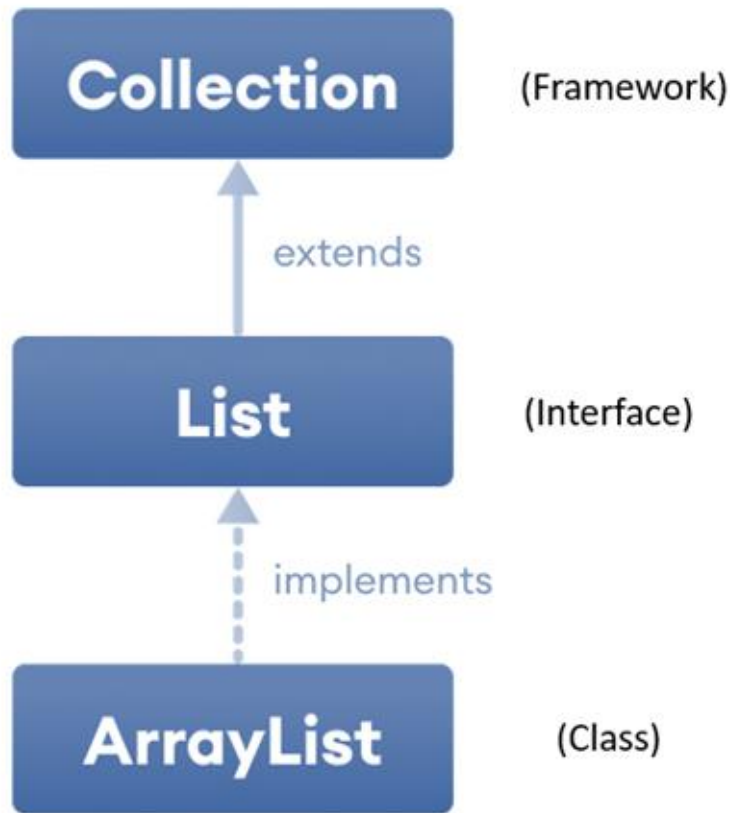
List and **ArrayList** are often used interchangeably, but they are **not the same**.

```
List<String> tasks = new ArrayList<>();
```



What is List?

- **List<E>** is an **interface** in Java that defines an ordered collection (a sequence).
- It provides abstract methods for adding, removing, and accessing elements.
- List **does NOT store data**; instead, it provides a contract that different implementations (like ArrayList and LinkedList) must follow.

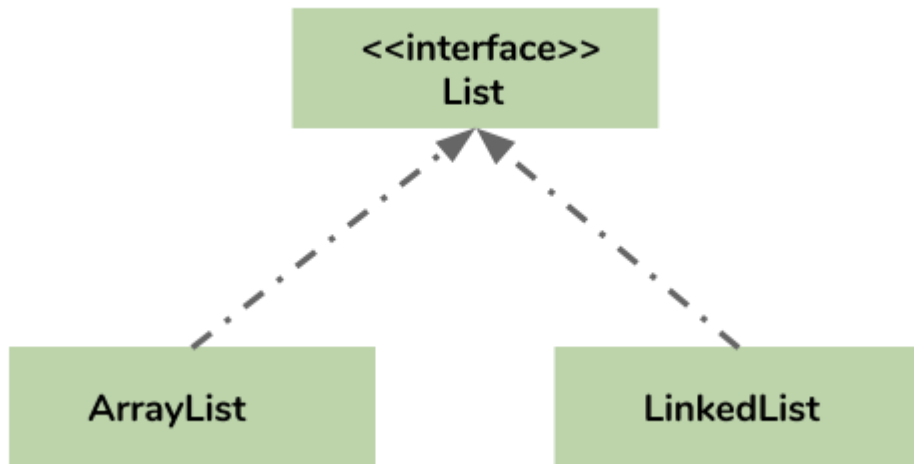


What is ArrayList?

ArrayList<E> is a **concrete implementation** of the List<E> interface.

It uses a dynamic array to store elements.

It provides fast random access ($O(1)$) but slow insertions/removals ($O(n)$).

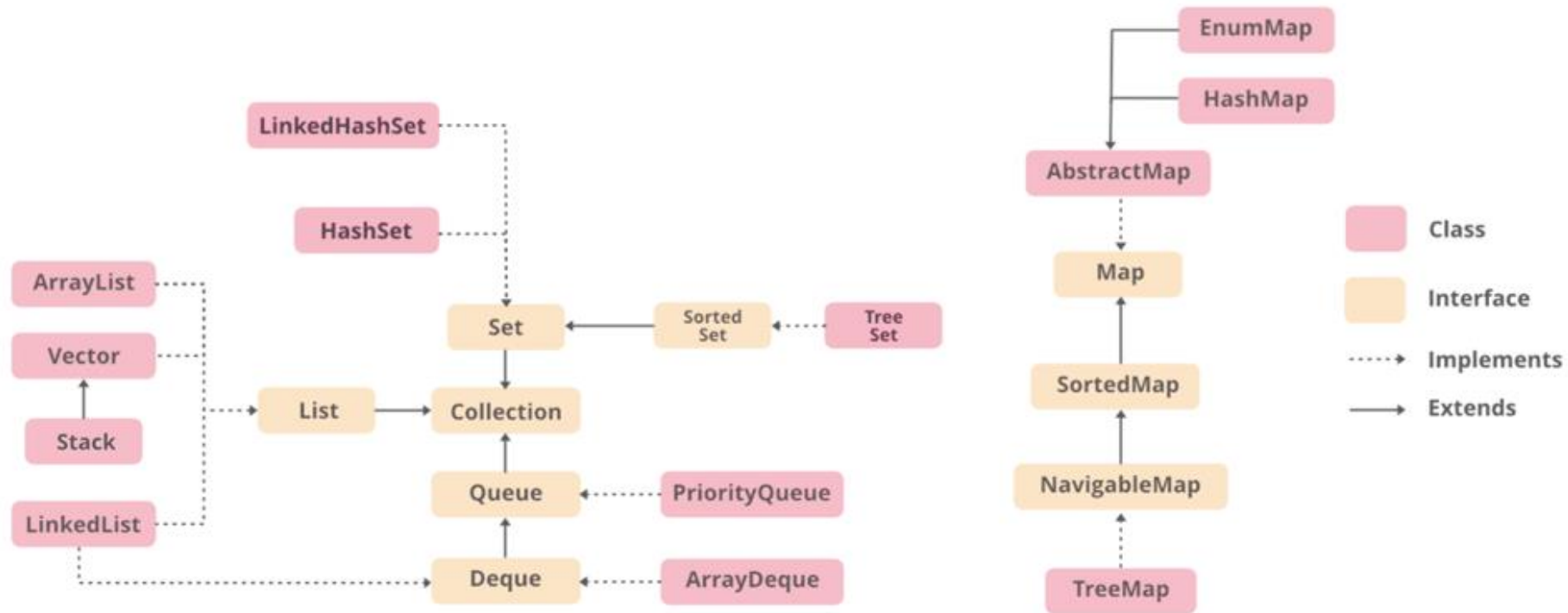


ArrayList

- fast random access
- slow insertion and deletion

LinkedList

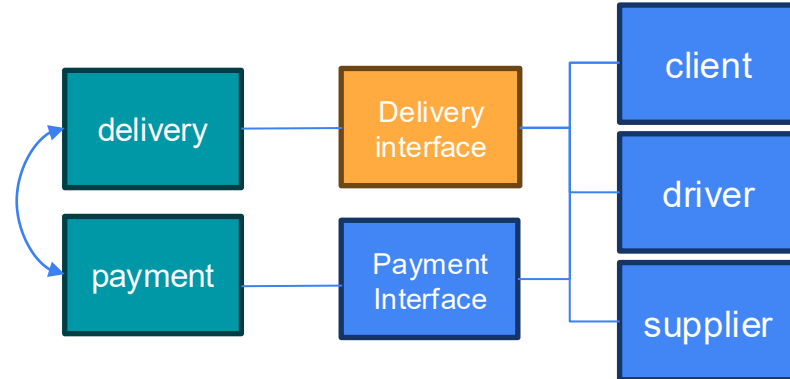
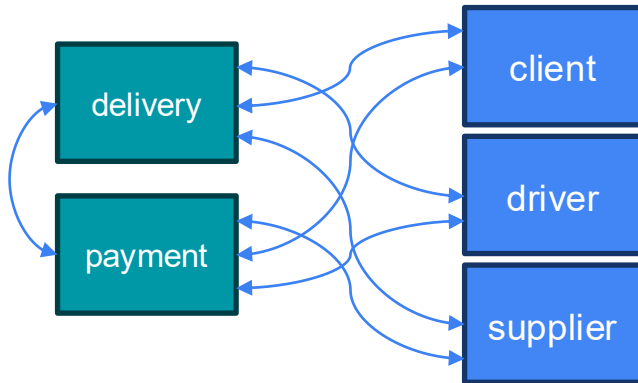
- fast insertion and deletion
- slow random access



Loose coupling

Loose coupling is a **design principle** that minimizes dependencies between components in a system.

When two classes or modules are loosely coupled, they **rely only on abstractions (interfaces or abstract classes)** rather than specific implementations. This makes the system more flexible, scalable, and maintainable.



Key Benefits of Loose Coupling

Easier to Modify – Changes in one class don't heavily impact others.

Improves **Code Reusability** – Components can be reused in different parts of the system.

Enhances **Testability** – Makes unit testing easier by allowing mock objects.

Supports the **Open/Closed Principle** – The system can be extended without modifying existing code.

Example of Tight vs. Loose Coupling

```
class PayPalPayment {
    void pay(double amount) {
        System.out.println("Paid using PayPal: " + amount);
    }
}

class OrderProcessor {
    private PayPalPayment payment;

    OrderProcessor() {
        this.payment = new PayPalPayment(); // Direct dependency
    }

    void processOrder(double amount) {
        payment.pay(amount);
    }
}
```

Problems:

Cannot switch payment methods easily (e.g., to Credit Card).
Any change in `PayPalPayment` affects `OrderProcessor`.
Violates the **Open/Closed Principle**.

Example of Tight vs. Loose Coupling

```
interface Payment {
    void pay(double amount);
}

class PayPalPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Paid using PayPal: " + amount);
    }
}

class CreditCardPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Paid using Credit Card: " + amount);
    }
}

class OrderProcessor {
    private Payment payment;

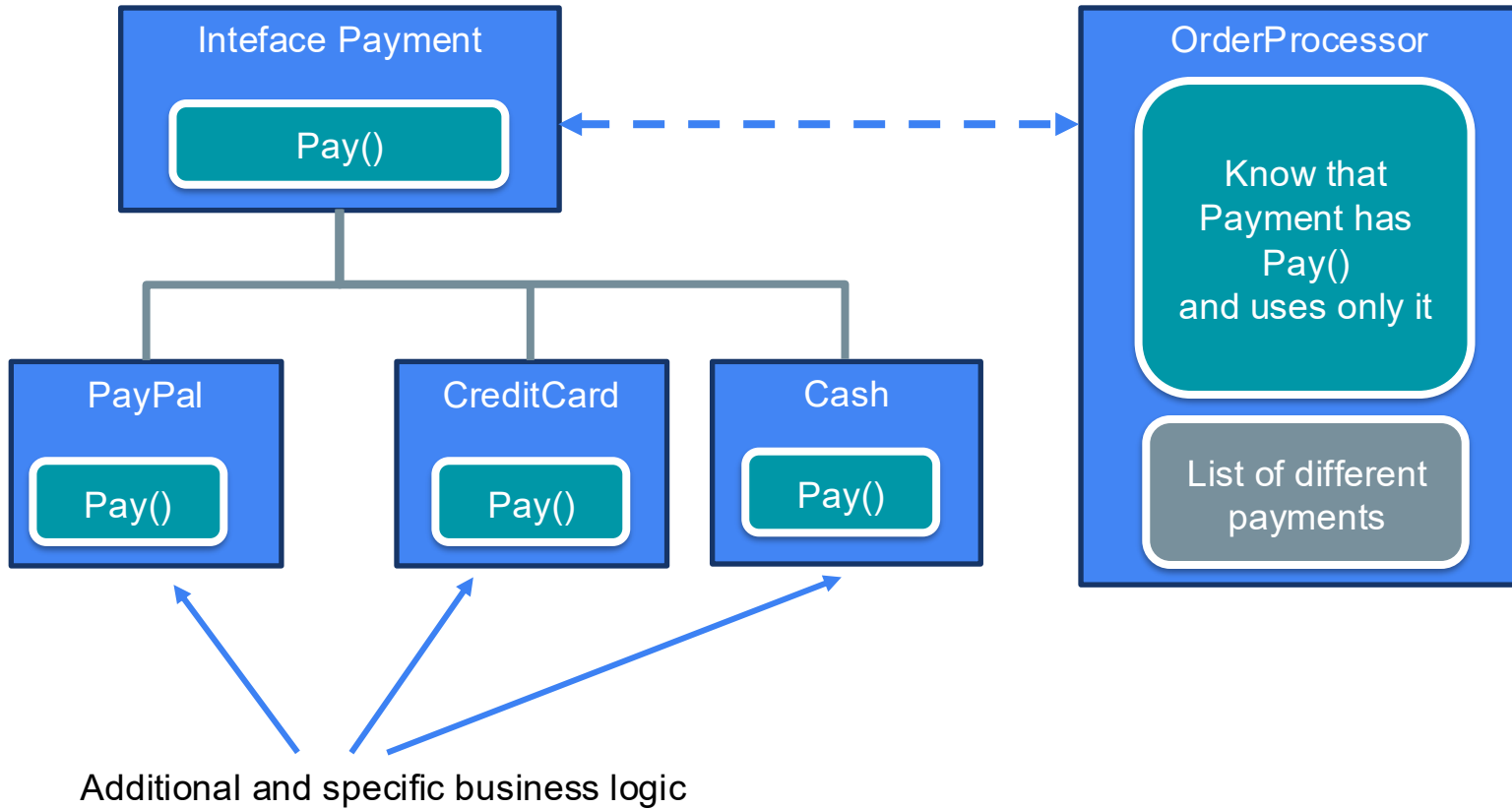
    OrderProcessor(Payment payment) { // Depend
        this.payment = payment;
    }

    void processOrder(double amount) {
        payment.pay(amount);
    }
}
```

Can easily switch between `PayPalPayment` and `CreditCardPayment` without modifying `OrderProcessor`.

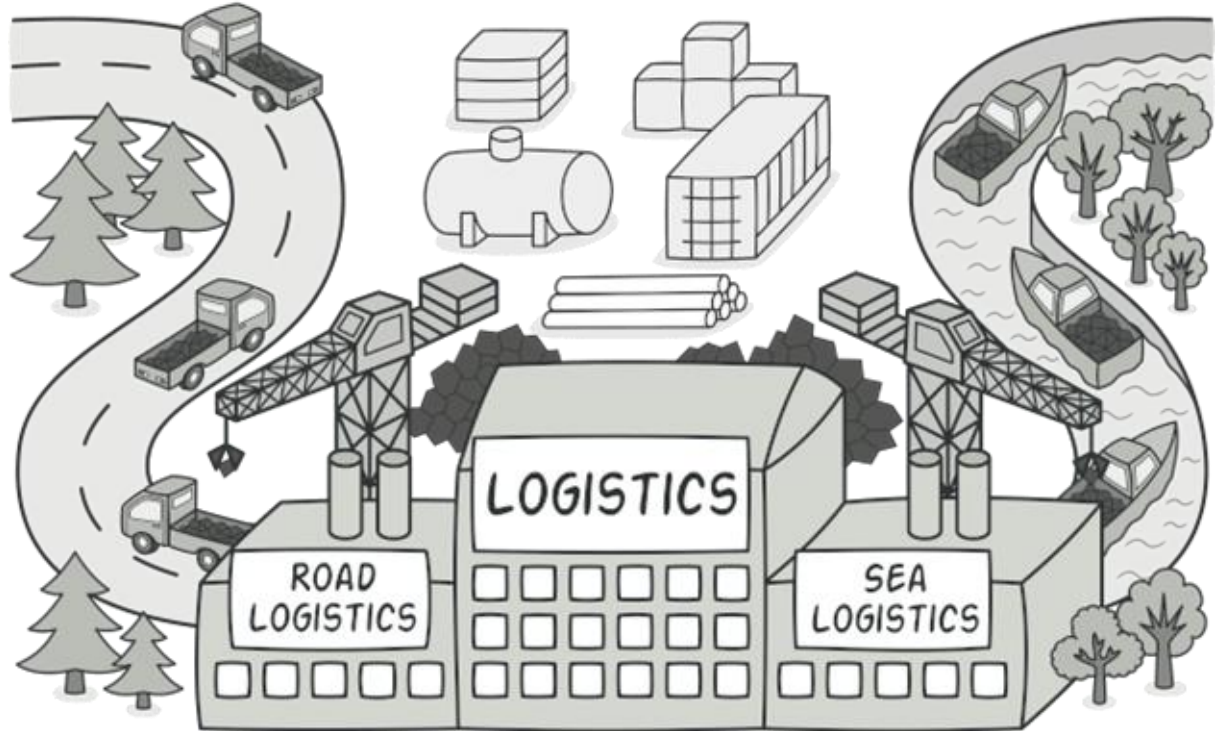
Future payment methods (e.g., Bitcoin) can be added **without modifying** existing classes.

`OrderProcessor` depends only on the **abstract Payment interface**, not on specific implementations.



Factory Method

Factory Method is a **creational design pattern** that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.



Problem Statement

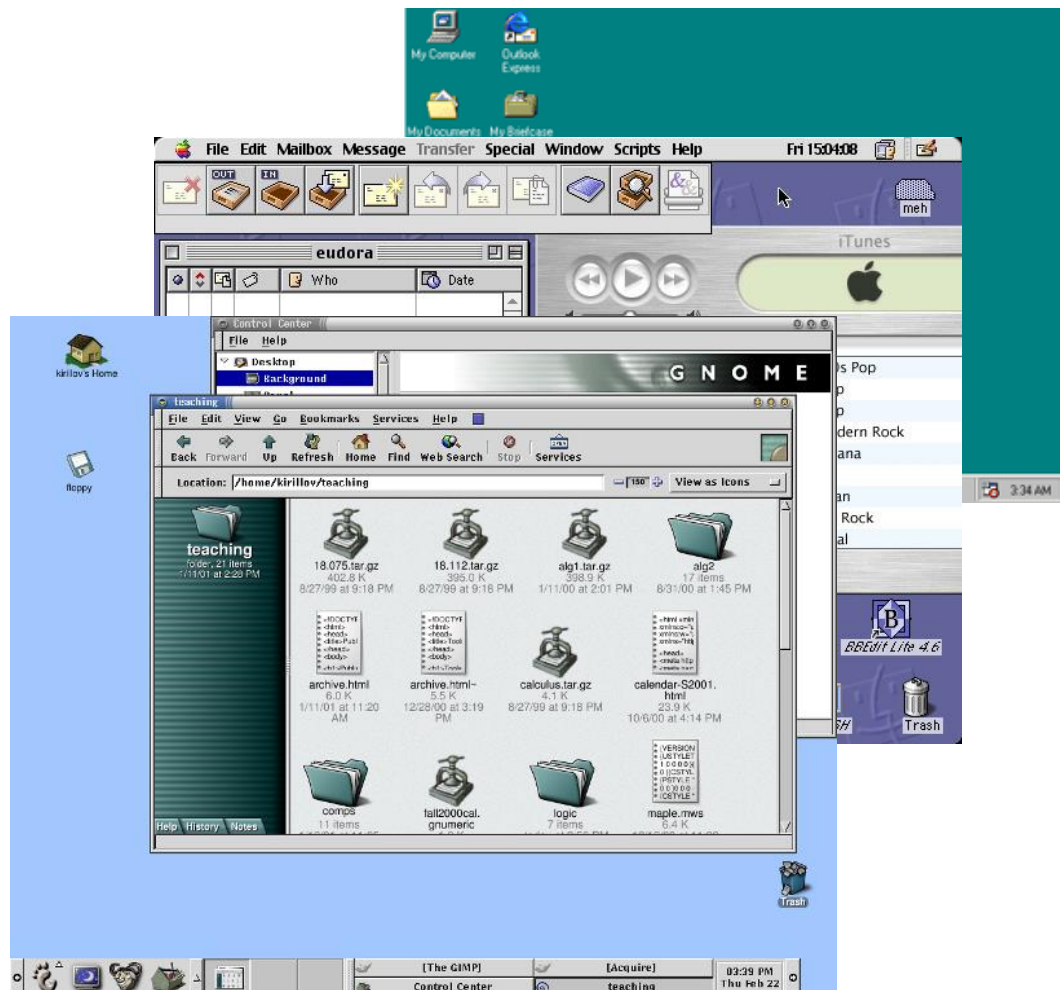
Imagine developing an app with different types of buttons:

- Windows Button
- MacOS Button
- Linux Button

Each button has different rendering logic, requiring conditional checks in the code.

This approach:

- Violates the Open/Closed Principle
- Makes maintenance difficult
- Leads to code duplication



Solution – Factory Method

Move object creation logic to a separate method (Factory Method).

Subclasses override this method to create specific objects.

Client code works with an abstract type, making it flexible and scalable.

Structure of the Factory Method

Product (Interface/Abstract Class)

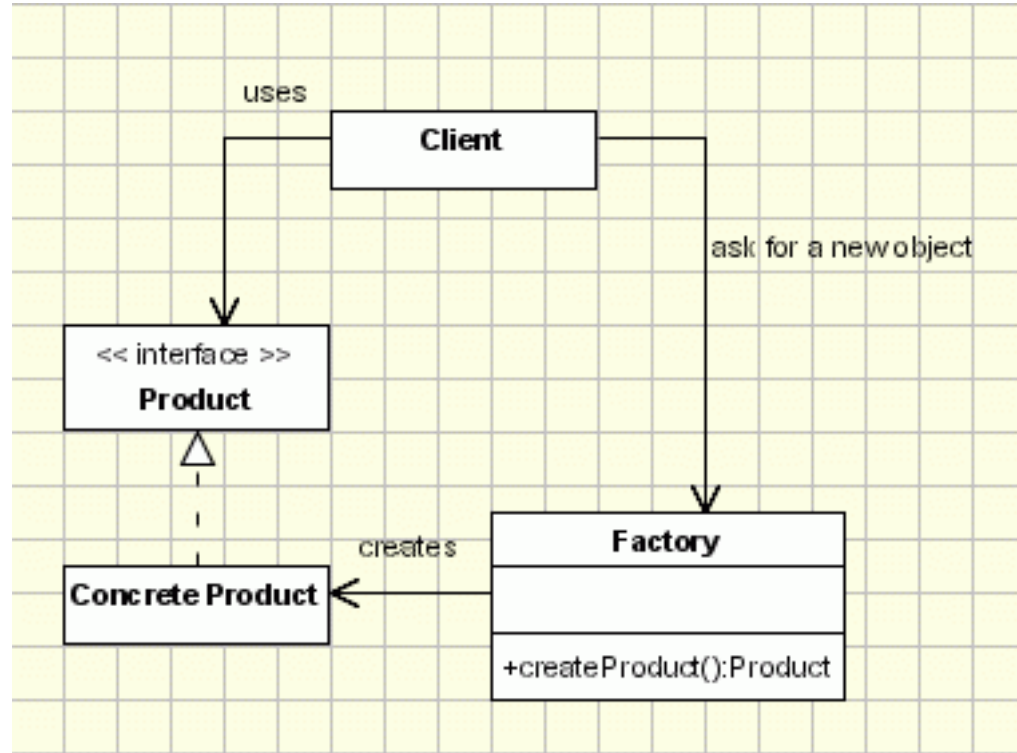
– Defines the common interface.

Concrete Products –

Implementations of the product.

Creator (Abstract Class) – Declares the Factory Method.

Concrete Creators – Override the Factory Method to instantiate specific products



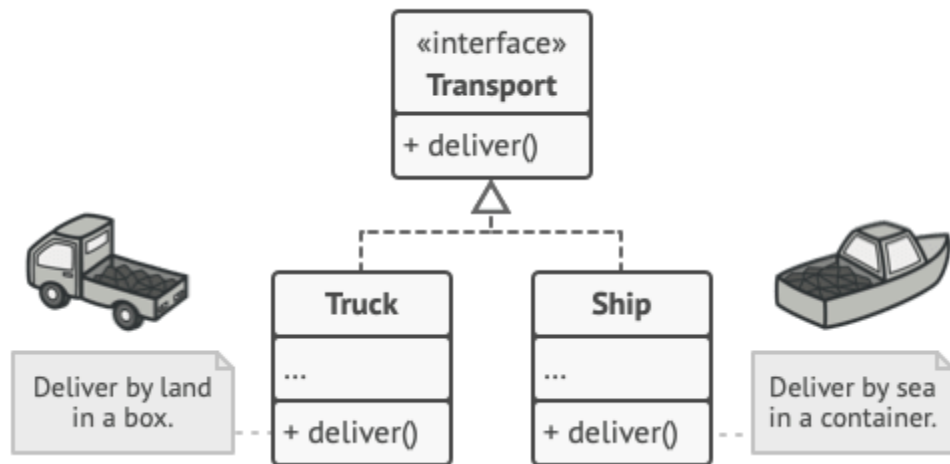


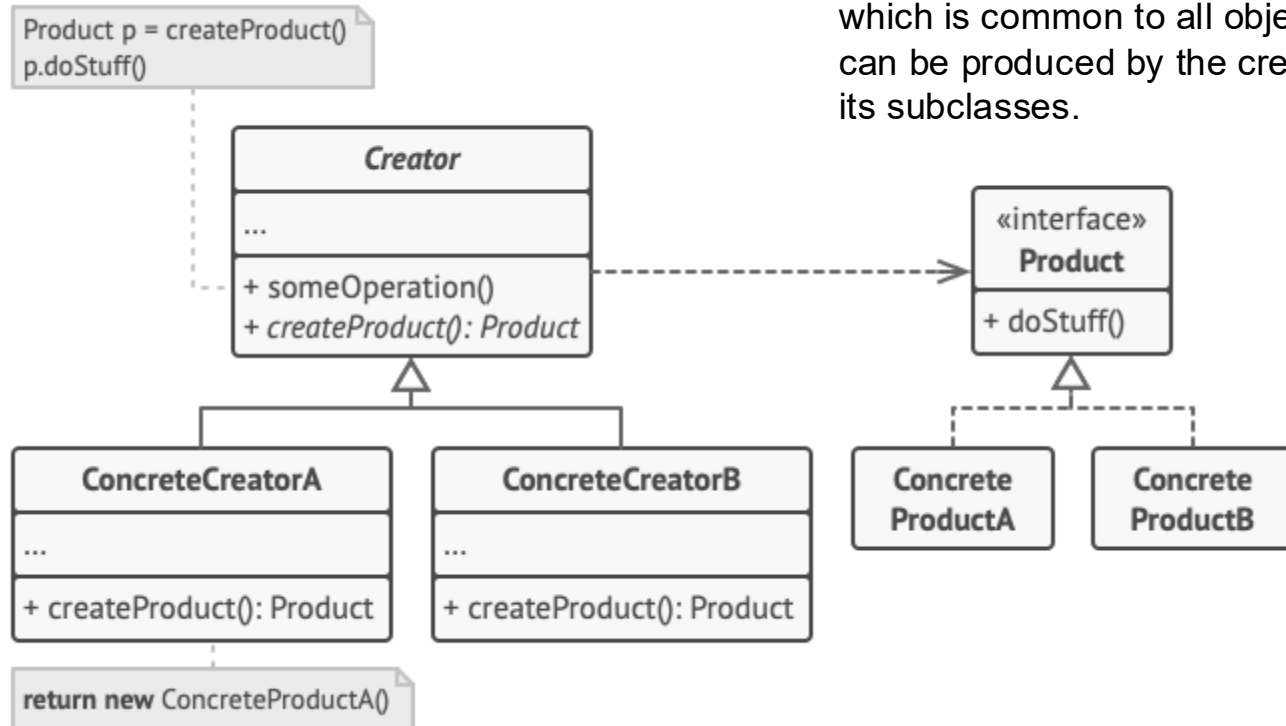
The Factory Method pattern suggests that you replace direct object construction calls (using the new operator) with calls to a special **factory method**.

the objects are still created via the **new** operator, but it's being called from within the factory method. Objects returned by a factory method are often referred to as products.

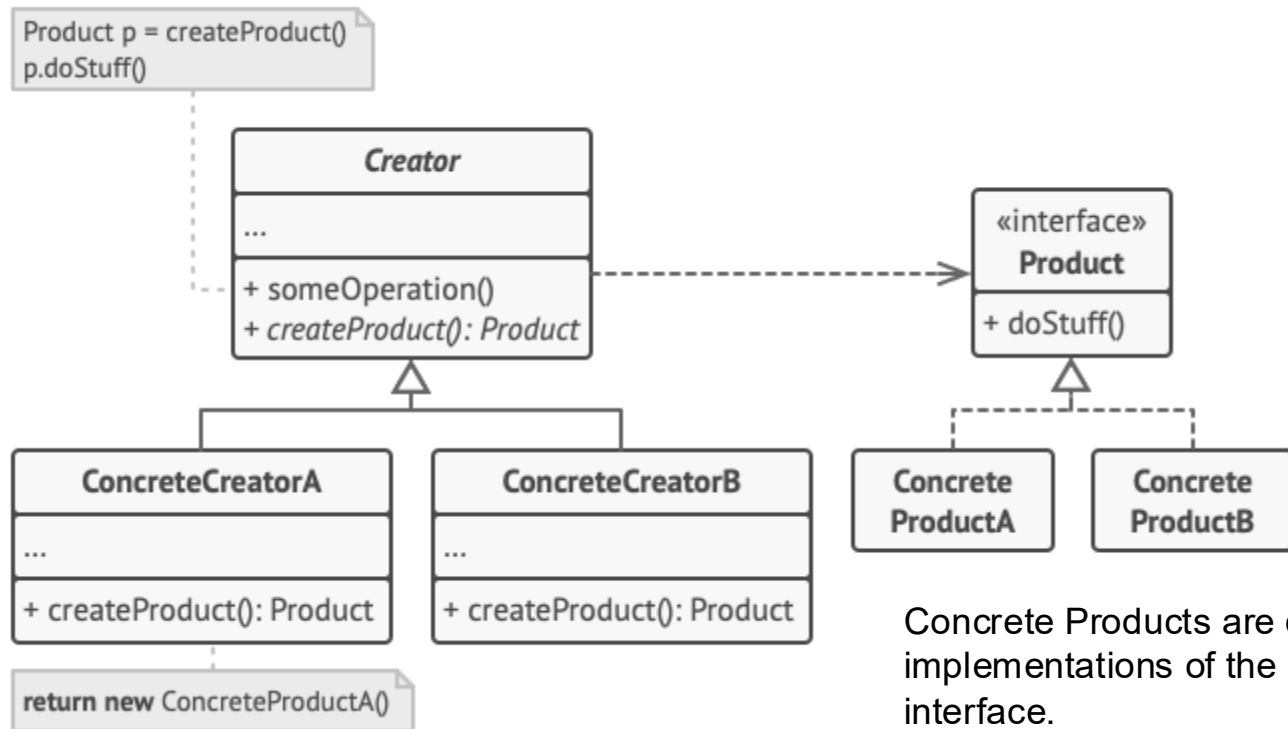
now you can override the factory method in a subclass and change the class of products being created by the method.

The code that uses the factory method (often called the **client** code) doesn't see a difference between the actual products returned by various subclasses. **The client treats all the products as abstract Transport.** The client knows that all transport objects are supposed to have the deliver method, but exactly how it works isn't important to the client.





The **Product** declares the interface, which is common to all objects that can be produced by the creator and its subclasses.

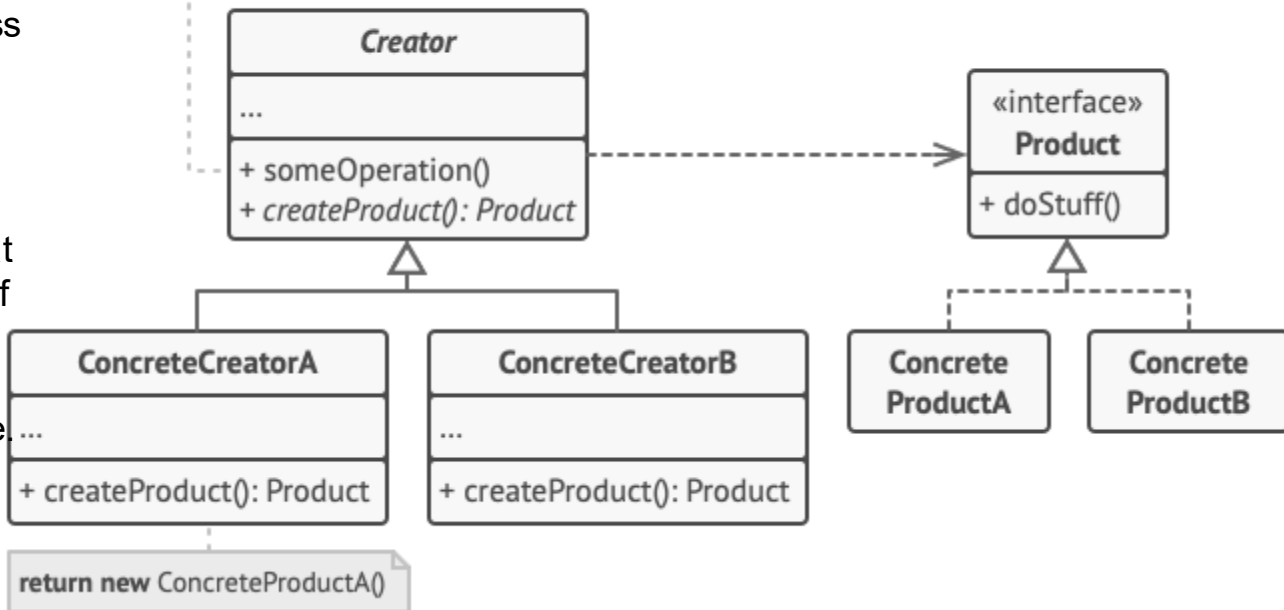


Concrete Products are different implementations of the product interface.

Note, despite its name, product creation is not the primary responsibility of the **creator**. Usually, the creator class already has some core business logic

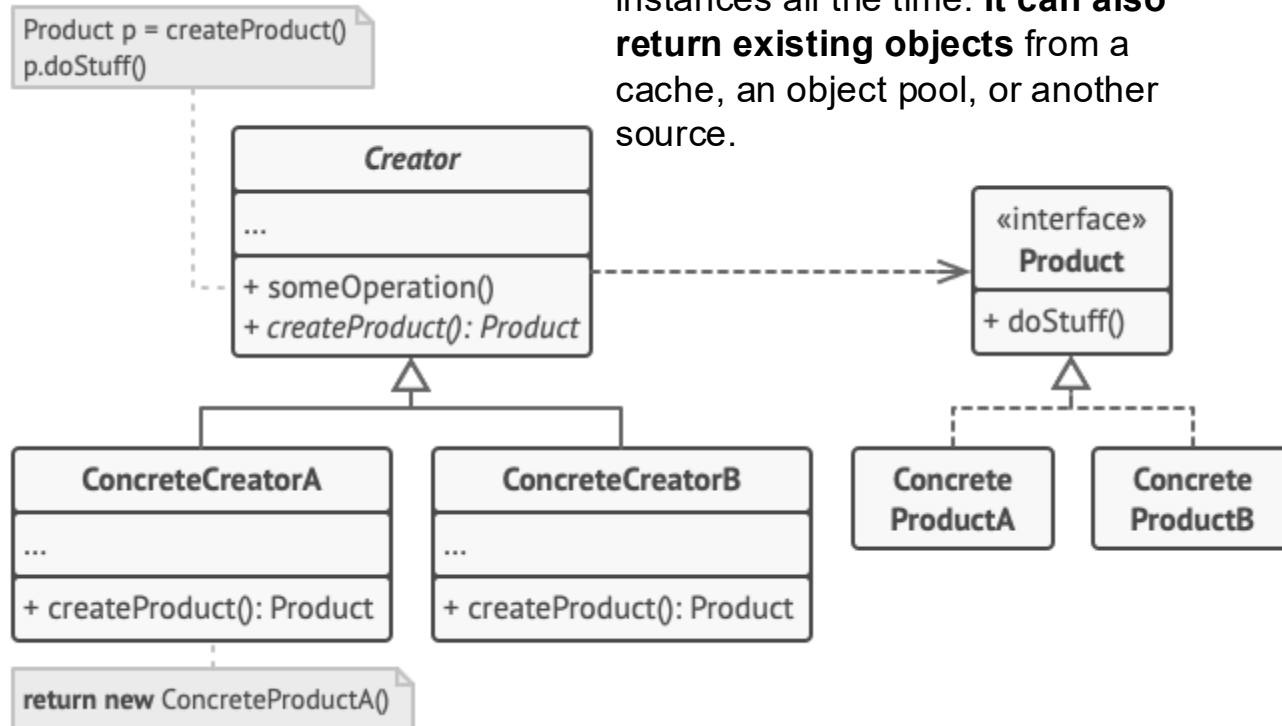
```
Product p = createProduct()
p.doStuff()
```

The Creator class declares the factory method that returns new product objects. It's important that the return type of this method matches the product interface.



Concrete Creators override the base factory method so it returns a different type of product.

Note that the factory method doesn't have to create new instances all the time. **It can also return existing objects** from a cache, an object pool, or another source.



Factory Method

The Factory Method **separates product construction code from the code that actually uses the product**. Therefore it's easier to extend the product construction code independently from the rest of the code.

For example, to add a new product type to the app, you'll only need to create a new creator subclass and override the factory method in it.

When to Use?

Use the Factory Method when you want to save system resources by reusing existing objects instead of rebuilding them each time.

Summary

You **avoid tight coupling** between the **creator** and the concrete **products**.

Single Responsibility Principle. You can move the product creation code into one place in the program, making the code easier to support.

Open/Closed Principle. You can introduce new types of products into the program without breaking existing client code.

The code may become more complicated since you need to introduce a lot of new subclasses to implement the pattern. The best case scenario is when you're introducing the pattern into an existing hierarchy of creator classes.

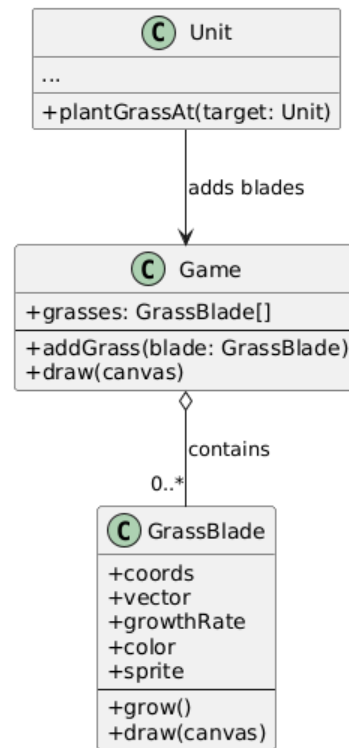


Statistics

Audio:
Level: -74.8 dB DSP load: 0.2%
Clipping: 0.0% Stream load: 0.0%

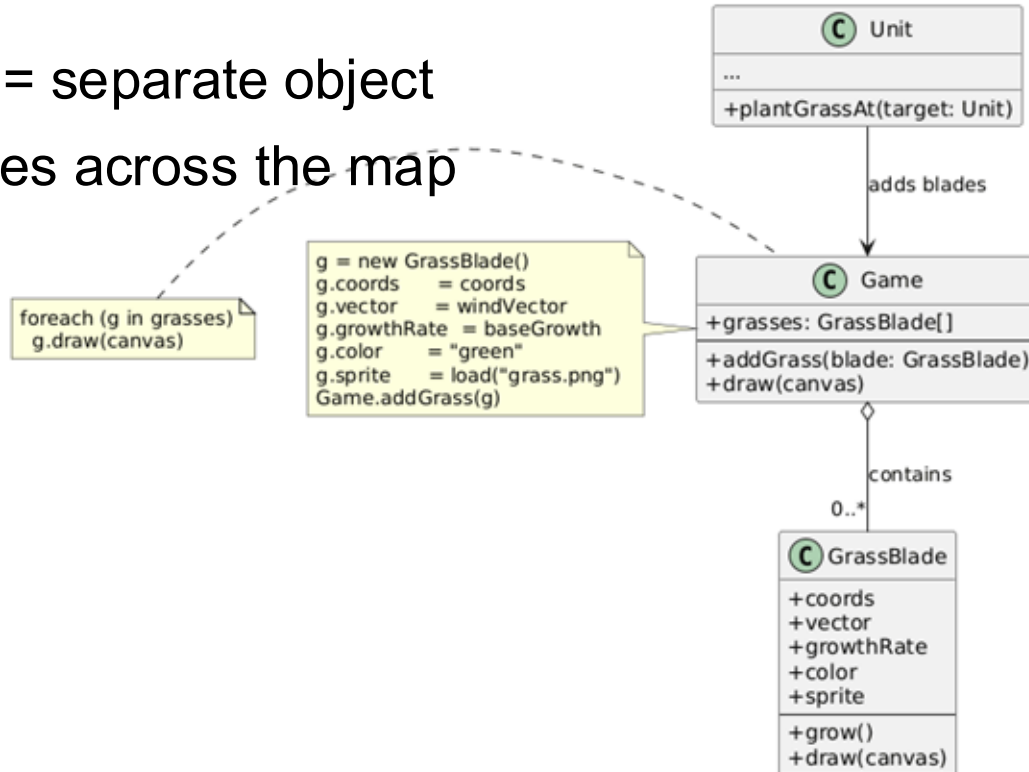
Graphics: 168.3 FPS (5.9ms)
CPU: main: 5.9ms render thread: 4.9ms
Batches: 10 Saved by batching: 813746
Tris: 11.3M Verts: 12.6M
Screen: 1548x571 - 15.4 MB
SetPass calls: 10 Shadow casters: 4
Visible skinned meshes: 0
Animation components playing: 0
Animator components playing: 0

UML



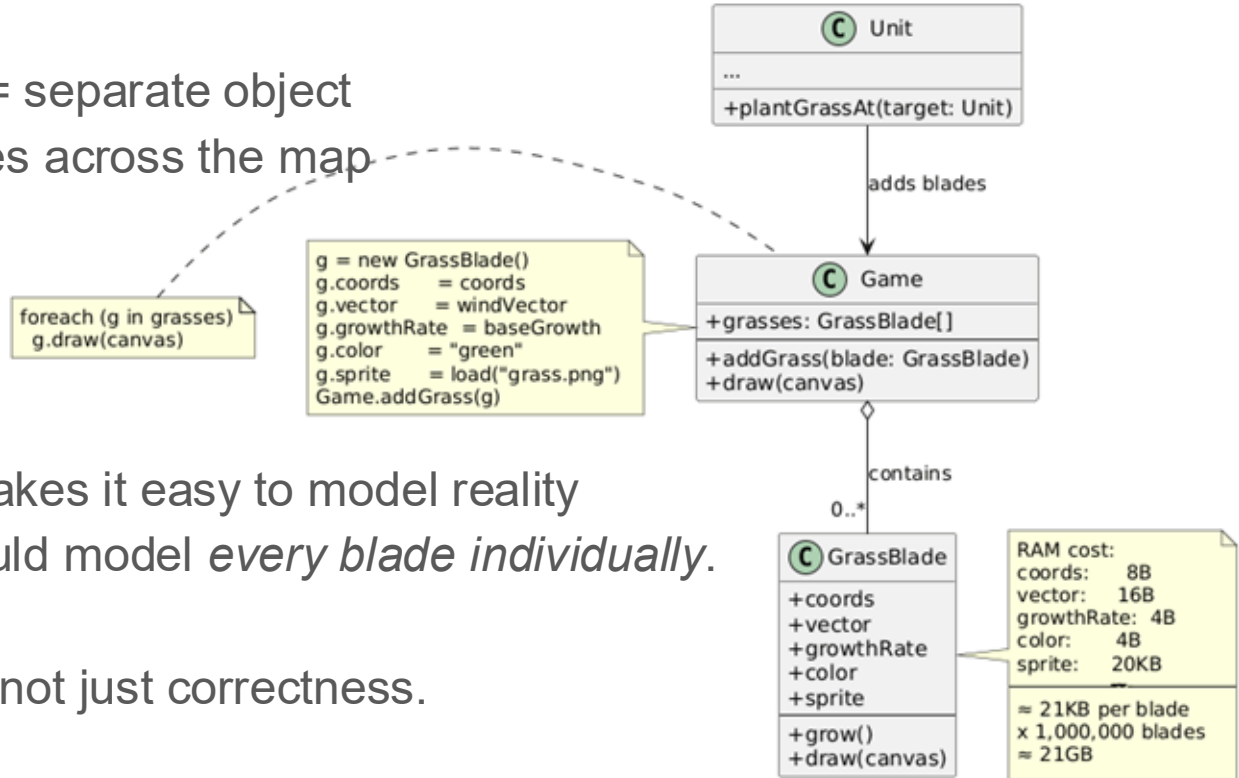
UML

- Every blade of grass = separate object
- Millions of grass blades across the map



UML

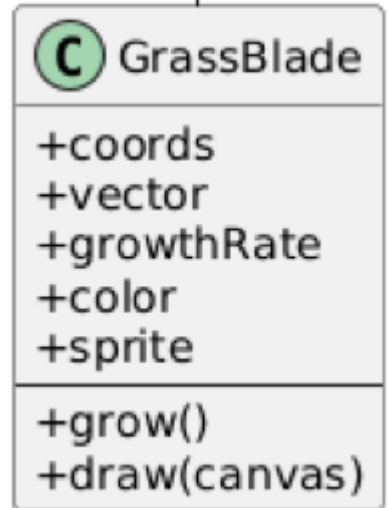
- Every blade of grass = separate object
- Millions of grass blades across the map



- Just because OOP makes it easy to model reality doesn't mean we should model *every blade individually*.
- Design for scalability, not just correctness.

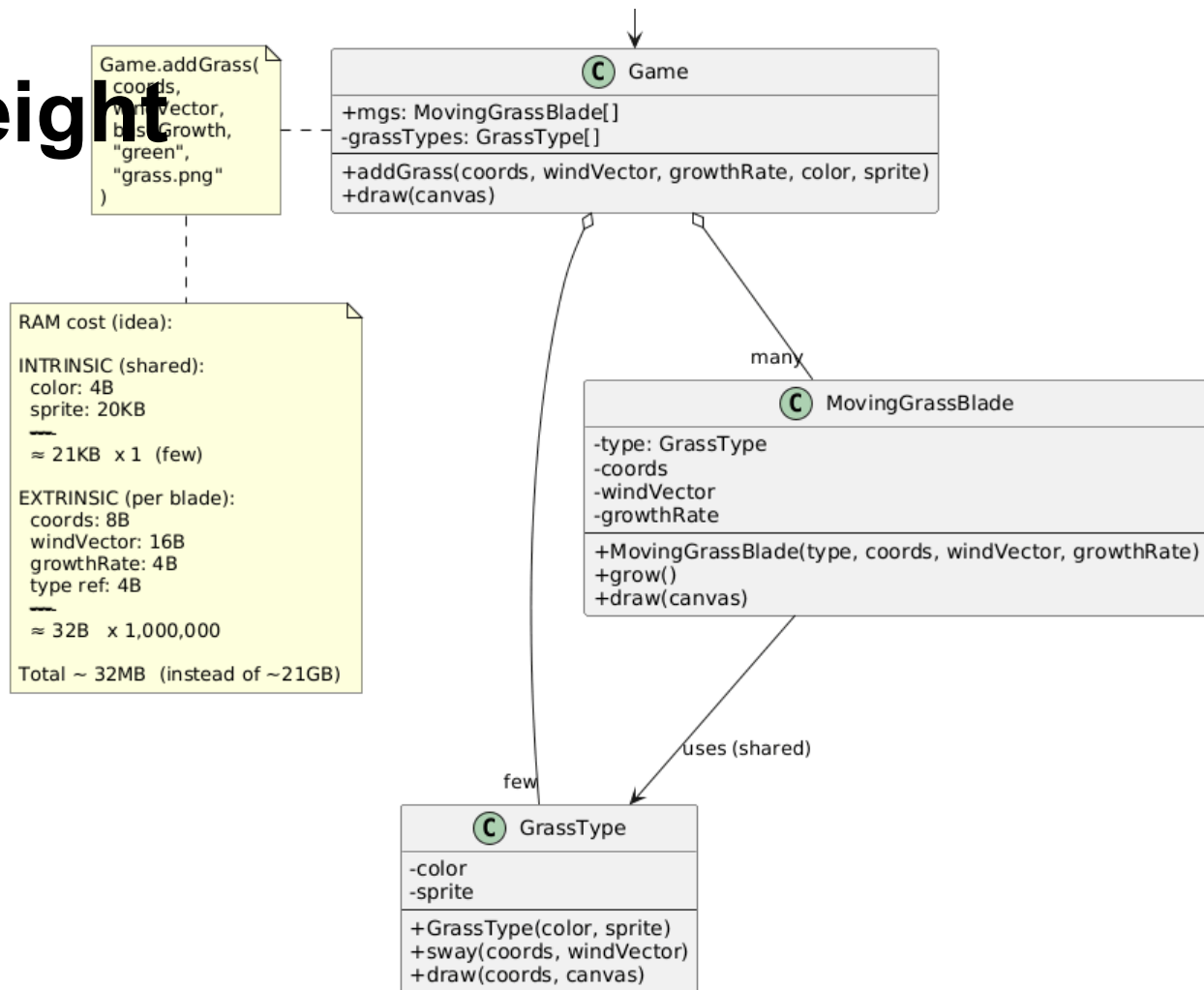
Problem

- *color* and *sprite* consume most of the memory
- But all grass blades share the same texture and color
- **Intrinsic** state (shared): *sprite*, *color*
- **Extrinsic** state (unique): *coords*, *wind vector*, *growthRate*
- We stored *both* inside every *GrassBlade* object
- Massive duplication

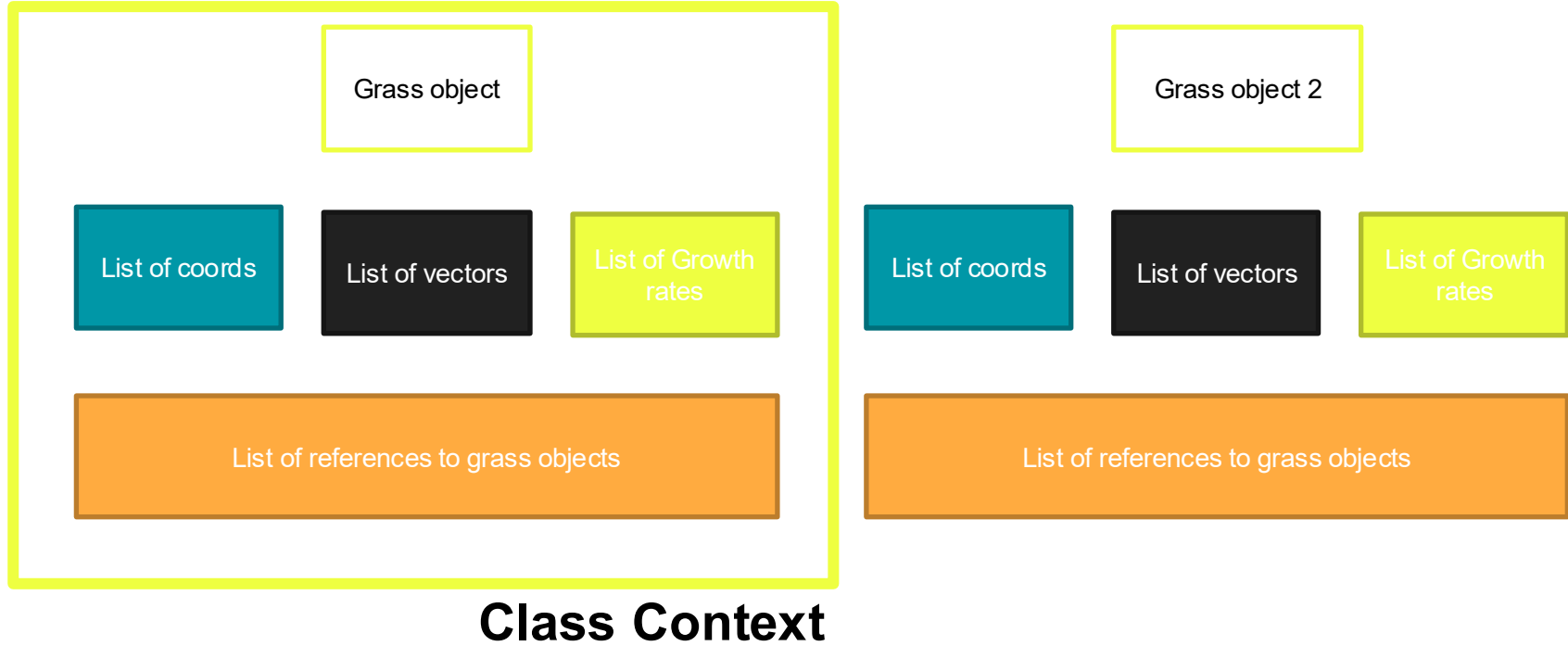


Solution - Flyweight

- Keep only **intrinsic (shared) state** inside the object
- Pass **extrinsic (changing) state** from outside (e.g., during draw)
- Reuse the same shared object across many instances



Extrinsic state storage



Extrinsic state storage

- **Extrinsic state** must still be stored somewhere
- Usually it moves to the **container** that holds many objects
- **Option A: “Raw arrays” in Game**
 - Separate arrays: coords[], windVectors[], growthRates[], plus typeRef[]
 - All arrays must stay perfectly **in sync** (same index = same blade)
 - Fast, but error-prone and ugly to maintain
- **Option B: Context objects (cleaner)**
 - Create a small MovingGrassBlade (context) that stores:
 - extrinsic state + reference to shared GrassType (flyweight)
 - Game keeps **one array**: MovingGrassBlade[]

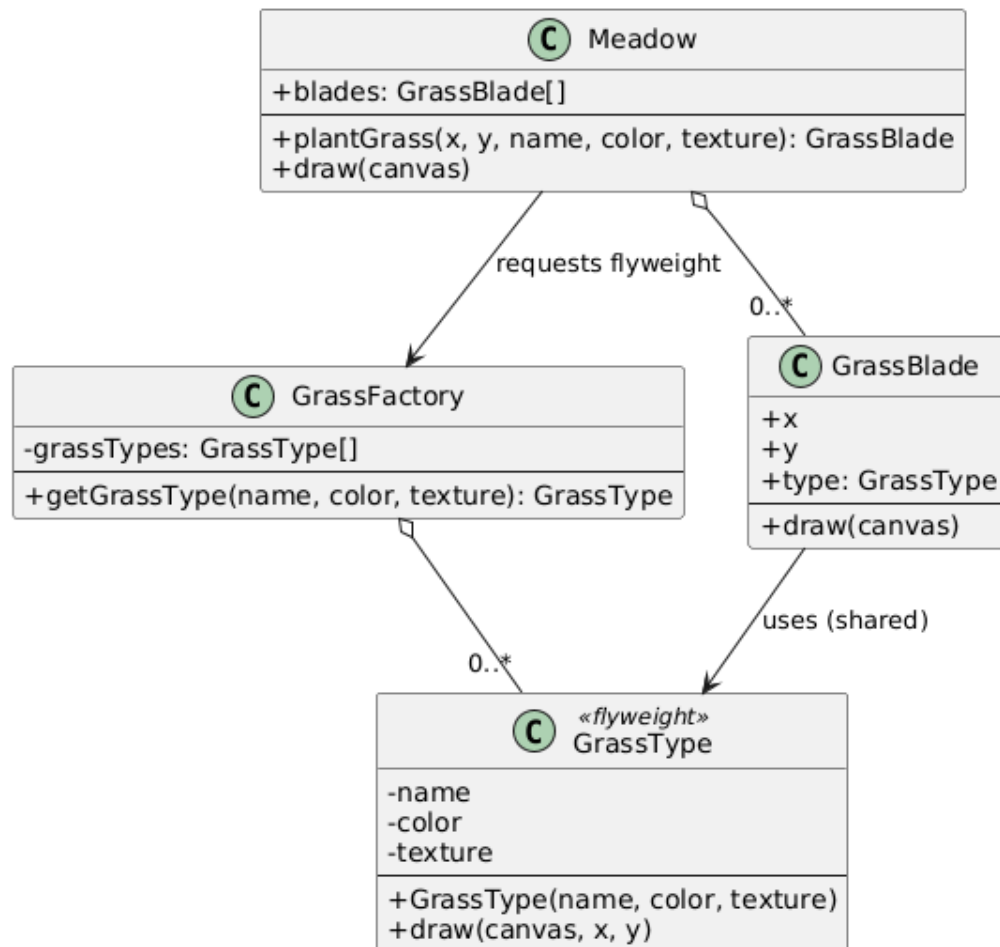




Flyweight Factory

- Use a factory to **reuse existing flyweights** instead of creating duplicates
- Input: intrinsic state (e.g., color + sprite)
- Factory logic:
 - look up matching GrassType in a pool
 - return it if found
 - otherwise create + store + return

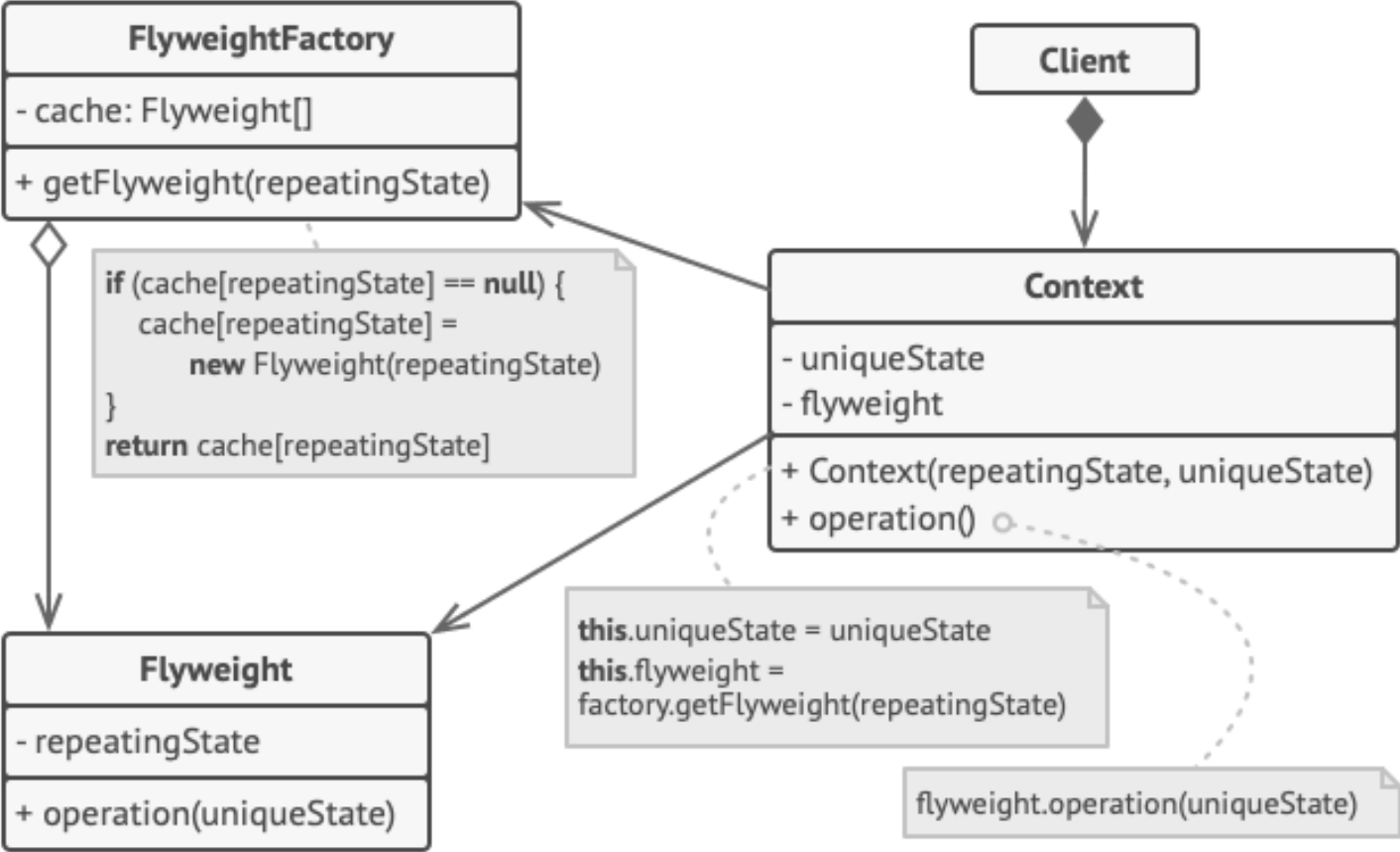
Flyweight for Grass (factory + container)



Flyweight needs immutability

- One GrassType (flyweight) is reused by **many** grass instances
- So its **intrinsic state must not change**
- initialize once via constructor
- **no setters**, no public mutable fields
- Otherwise changing one flyweight would unexpectedly affect *all* blades using it

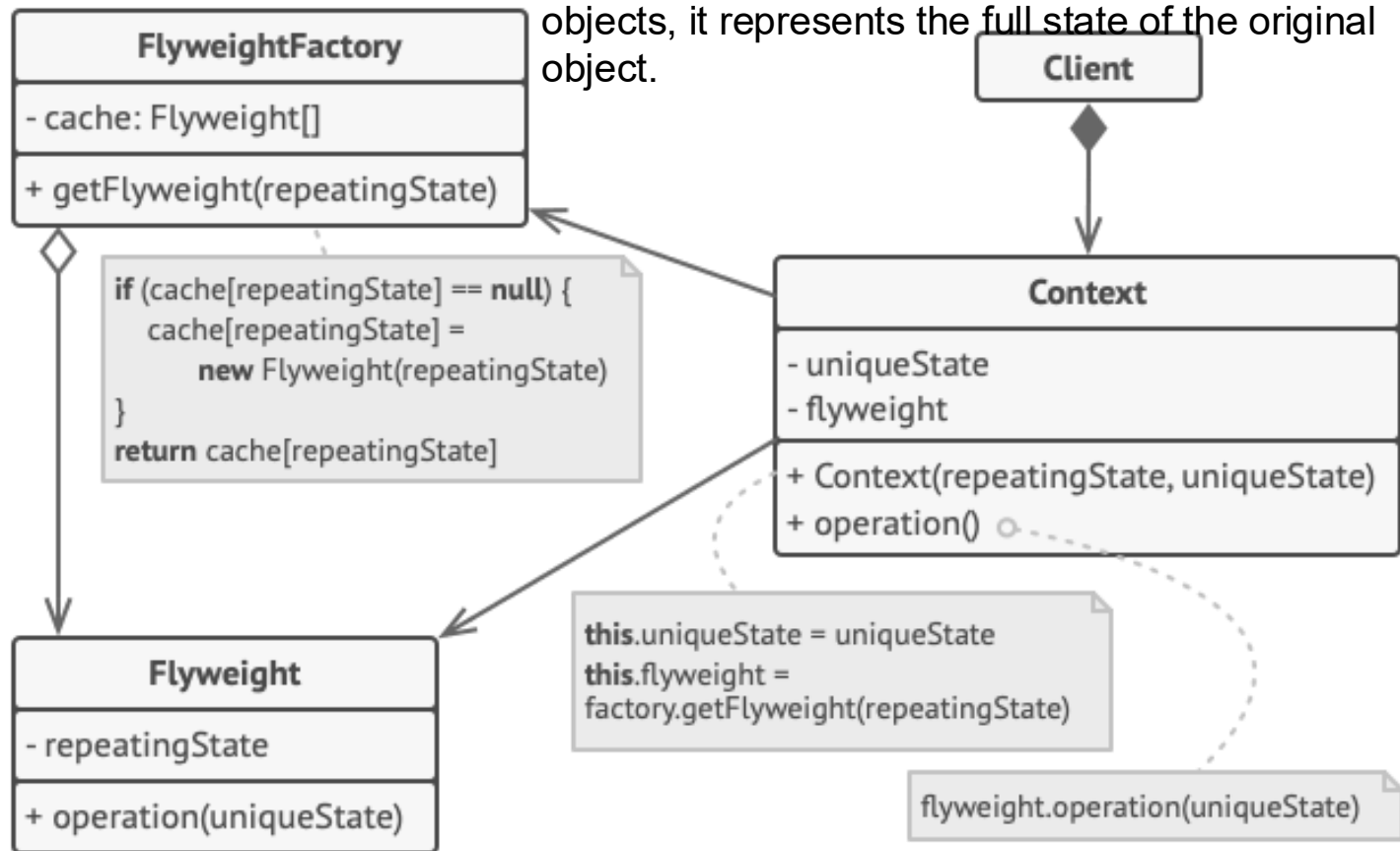
Structure



The **Flyweight** class contains the portion of the original object's state that can be shared between multiple objects.

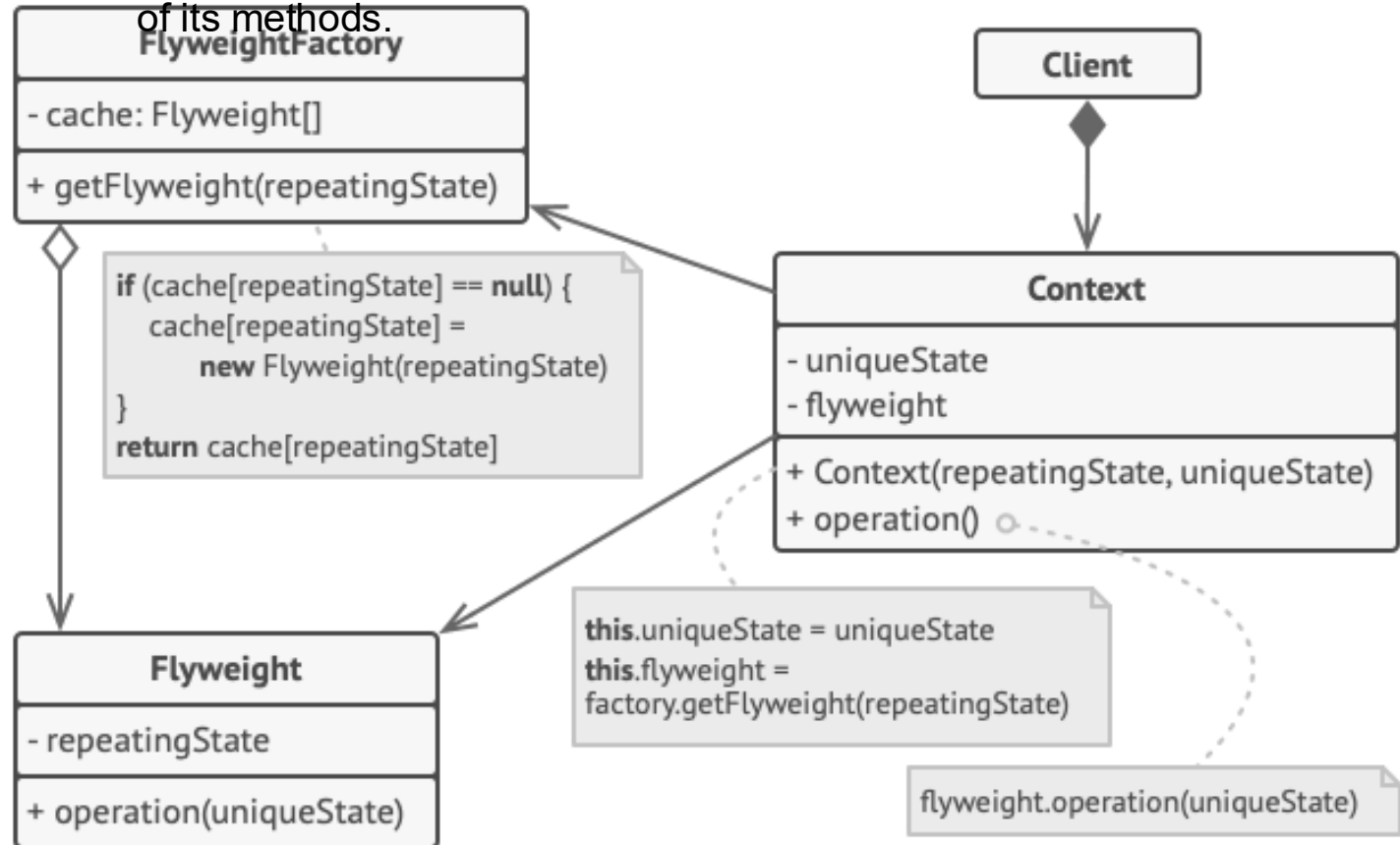
Structure

The **Context** class contains the extrinsic state, unique across all original objects. When a context is paired with one of the flyweight objects, it represents the full state of the original object.



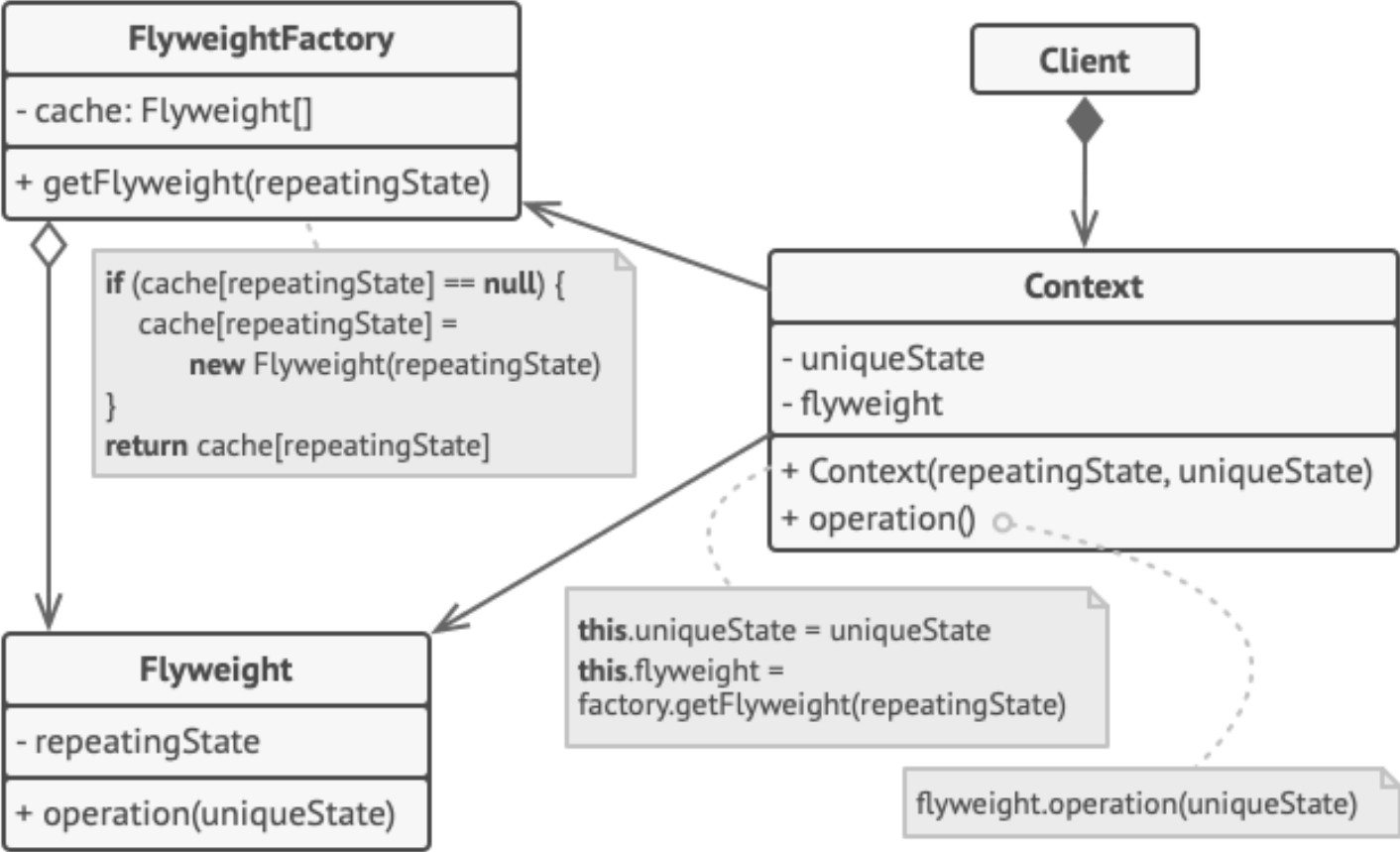
Structure

The **Client** calculates or stores the extrinsic state of flyweights. From the client's perspective, a flyweight is a template object which can be configured at runtime by passing some contextual data into parameters of its methods.



Structure

The **Flyweight** **Factory** manages a pool of existing flyweights. With the factory, clients don't create flyweights directly. Instead, they call the factory, passing it bits of the intrinsic state of the desired flyweight. The factory looks over previously created flyweights and either returns an existing one that matches search criteria or creates a new one if

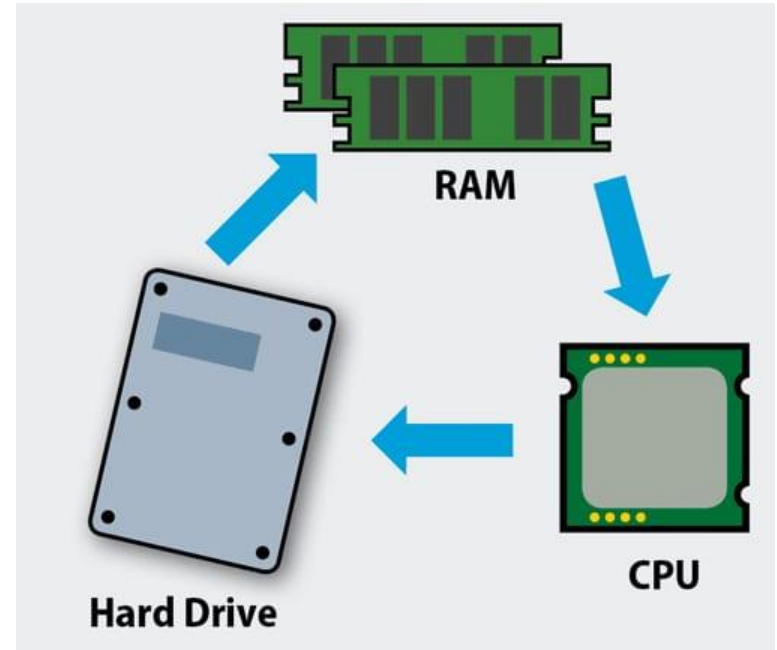


Applicability

- The Flyweight pattern is merely an optimization.
- Before applying it, make sure your program does have the RAM consumption problem related to having a massive number of similar objects in memory at the same time.
- Make sure that this problem can't be solved in any other meaningful way.
- useful when:
 - an application needs to spawn a huge number of similar objects
 - this drains all available RAM on a target device
 - the objects contain duplicate states which can be extracted and shared between multiple objects

Disadvantages

- You might be trading **RAM over CPU** cycles when some of the context data needs to be recalculated each time somebody calls a flyweight method.
- The code becomes much more **complicated**. New team members will always be wondering why the state of an entity was separated in such a way.



Flyweight can resemble Singleton

- if you somehow managed to reduce all shared states of the objects to just one flyweight object
- fundamental differences:
 - There should be only one Singleton instance
 - a *Flyweight* class can have multiple instances with different intrinsic states
 - The *Singleton* object can be mutable
 - Flyweight objects are immutable.